Assignment 3: Understanding Algorithm Efficiency and Scalability

Fathiya Adan

University of Cumberlands

Algorithms and Data Structures (MSCS-532-B01)

Michael Solomon

March 22, 2026

1. Introduction

Algorithm efficiency and scalability are fundamental concepts in computer science, particularly when designing systems that must handle large amounts of data. The performance of an algorithm is often influenced not only by its theoretical complexity but also by implementation choices and input characteristics.

This assignment focuses on two important topics:

1. Randomized Quicksort versus Deterministic Quicksort
2. Hash Tables with Chaining

The objective is to analyze these algorithms both theoretically and empirically, highlighting how design decisions such as pivot selection and collision handling impact performance.

2. Part 1 – Randomized Quicksort Analysis

2.1 Implementation Overview

Two versions of Quicksort were implemented in Python:

- **Randomized Quicksort:** selects a pivot randomly from the array
- **Deterministic Quicksort:** always selects the first element as the pivot

Both implementations use the **Lomuto partition scheme** and recursively sort the left and right subarrays.

The algorithms were tested on four types of input data:

- Random arrays
- Sorted arrays
- Reverse-sorted arrays
- Arrays with repeated elements

Execution time was measured using Python's time module.

2.2 Theoretical Analysis

The running time of Quicksort depends on how balanced the partitions are.

Average Case (Randomized Quicksort)

The expected running time is:

[
$T(n) = O(n \log n)$
]

This occurs because:

- The pivot is randomly selected
- The probability of balanced partitions is high
- The recursion depth is approximately ($O(\log n)$)

Worst Case

The worst case occurs when partitions are highly unbalanced:

[
$T(n) = T(n-1) + O(n) = O(n^2)$
]

This happens when:

- The smallest or largest element is always chosen as pivot
- This is common in deterministic Quicksort on sorted inputs

2.3 Experimental Results

The following results were obtained from running the algorithms:

Selected Results from Execution

Size	Distribution	Randomized QS (s)	Deterministic QS (s)
100	Random	0.000429	0.000085
100	Sorted	0.000158	0.000375
100	Reverse Sorted	0.000110	0.000514
500	Sorted	0.001003	0.005729
1000	Sorted	0.001740	0.021404
1000	Reverse Sorted	0.001335	0.040108
2000	Sorted	0.003565	0.083424

2000 Reverse Sorted	0.003650	0.156437
5000 Sorted	0.009371	0.518809
5000 Reverse Sorted	0.011625	0.981448
5000 Random	0.010735	0.008723
5000 Repeated Elements	0.288222	0.290561

2.4 Analysis of Results

The experimental data clearly supports the theoretical expectations.

1. Random Inputs

For random arrays:

- Randomized QS (5000): 0.010735 s
- Deterministic QS (5000): 0.008723 s

Both algorithms perform similarly, confirming that:

- Deterministic pivot selection is acceptable for random inputs
- Both behave close to (O(n \log n))

2. Sorted Inputs (Critical Case)

For sorted arrays:

- At size 5000:
    - Randomized QS: 0.009371 s
    - Deterministic QS: 0.518809 s

This shows that deterministic Quicksort is:

[
\approx 55 \text{ times slower}
]

This occurs because:

- The first element is always the smallest

- Each partition becomes extremely unbalanced
- The recursion becomes linear instead of logarithmic

3. Reverse-Sorted Inputs

For reverse-sorted arrays:

- At size 5000:
    - Randomized QS: 0.011625 s
    - Deterministic QS: 0.981448 s

This is an even worse case:

$$
\approx 84 \text{ times slower}
$$

This confirms the worst-case behavior of deterministic Quicksort.

4. Repeated Elements

For repeated values:

- At size 5000:
    - Randomized QS: 0.288222 s
    - Deterministic QS: 0.290561 s

Both algorithms perform similarly but slower than random inputs.

Reason:

- Partitioning is less efficient when many values are equal
- Standard two-way partitioning struggles with duplicates

2.5 Key Insight

Randomized Quicksort provides consistent and reliable performance, while deterministic Quicksort is highly sensitive to input structure. This demonstrates the importance of randomization in algorithm design.

3. Part 2 – Hash Table with Chaining

3.1 Implementation Overview

A hash table was implemented using **chaining** to handle collisions. The table consists of an array where each index contains a list (bucket) of key-value pairs.

The implementation supports:

- Insert operation
- Search operation
- Delete operation

A hash function maps keys to indices using:

$$
\text{index} = \text{hash(key)} \mod \text{table size}
$$

3.2 Collision Handling

Collisions occur when multiple keys map to the same index. In this implementation, collisions are handled using chaining, where each bucket stores multiple elements.

Example:

Index 2 → [(apple, 10), (grape, 40)]

This approach ensures that no data is lost when collisions occur.

3.3 Load Factor

The load factor is defined as:

$$
\alpha = \frac{n}{m}
$$

Where:

- (n) is the number of stored elements
- (m) is the number of buckets

The load factor determines how full the table is.

3.4 Resizing Strategy

To maintain efficiency, the table resizes when the load factor exceeds 0.75. The resizing process involves:

- Doubling the table size
- Rehashing all existing elements

Although resizing takes (O(n)) time, it occurs infrequently, so the amortized cost of insertion remains (O(1)).

3.5 Time Complexity Analysis

Average Case

- Insert: (O(1))
- Search: (O(1))
- Delete: (O(1))

These assume a good hash function and controlled load factor.

Worst Case

- Insert: (O(n))
- Search: (O(n))
- Delete: (O(n))

This occurs when all elements are stored in the same bucket.

3.6 Experimental Verification

The implementation was tested using sample key-value pairs. The results confirmed:

- Successful insertion and retrieval of values
- Correct deletion of elements
- Accurate load factor calculation

For example, after deleting one element from four inserted values, the load factor was correctly updated to 0.3.

4. Conclusion

This assignment highlights how algorithm design decisions significantly impact performance. Randomized Quicksort demonstrates consistent and efficient behavior across various input types, while deterministic approaches can degrade dramatically in structured cases.

Similarly, hash tables with chaining provide efficient average-case performance for data storage and retrieval. However, their efficiency depends on maintaining a balanced load factor and effectively handling collisions.

Overall, the results emphasize the importance of combining theoretical analysis with practical implementation to design robust and scalable algorithms.

5. References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Python*. Wiley.